

Arbitrary attributes in `define_aggregate`

Document #: P3678R0 [[Latest](#)] [[Status](#)]
Date: 2025-05-15
Project: Programming Language C++
Audience: SG7
Reply-to: Aurelien Cassagnes
<acassagnes@bloomberg.net>

Contents

- [1 Introduction](#)
 - [1.1 Data member options](#)
- [2 Proposal](#)
- [3 Proposed wording](#)
 - [3.1 Library](#)
- [4 Implementation experience](#)
- [5 References](#)

§ 1 Introduction

The reflection paper ([\[P2996R10\]](#)) introduces `define_aggregate` as a lightweight construct to generate aggregates. We feel the way it is accomplished, while it satisfies some need, does scale poorly. We propose in this paper a generic way to support attributes via `define_aggregate` and the attributes reflection offered in [\[P3385R4\]](#)

§ 1.1 Data member options

The non static data members specification for `define_aggregate` are carried via `data_member_options` whose detail we reproduce here

```
struct data_member_options {  
    struct name_type {  
        template <typename T> requires constructible_from<u8string, T>  
            consteval name_type(T &&);  
  
        template <typename T> requires constructible_from<string, T>  
            consteval name_type(T &&);  
    };  
};
```

```

optional<name_type> name;
optional<int> alignment;
optional<int> bit_width;
bool no_unique_address = false;
};

```

Note here that two standard attributes are supported: `alignment` and `no_unique_address`. No special rationale are given as to why those were picked out, one guess as the fact that they actually affect the program layout and potentially its behaviour... so in the end maybe there lies the rationale. Ultimately and more realistically, the biggest culprit is the lack of generic way to carry attributes around. A design that crowds `data_member_options` with `is_nodiscard`, `is_deprecated`, etc. would have been equally unsatisfying. Since [\[P3385R4\]](#) is making progress towards standardization, we feel now is an appropriate time to address that design quirk.

§ 2 Proposal

We propose that attributes for the data member being specified be done transparently via a dedicated attributes data member, enumerating all attributes to appertain to member being specified. The change to the `data_member_options` structure looks as such

```

namespace std::meta {
    struct data_member_options {
        struct name_type {
            template <typename T> requires constructible_from<u8string, T>
                consteval name_type(T &&);

            template <typename T> requires constructible_from<string, T>
                consteval name_type(T &&);
        };

        optional<name_type> name;
        optional<int> alignment;
        optional<int> bit_width;
-       bool no_unique_address = false;
+       [[deprecated]] bool no_unique_address = false;
+       vector<info> attributes;
    };
}

```

Some decisions are worth discussing

- Deprecating the `no_unique_address` field: Having a denormalized way to indicate that the member under specification is `no_unique_address` is uncomfortable and confusing, so we think deprecating this is the right procedure. As no current production code is being written at the moment that relies on this, it is also the proper time to address it.

- Leaving alignment as is: While *alignment-specifier* are technically considered attributes, their form predates CXX11 standard way to prescribe those. As a result the alignment attribute is left out of the scope of what [P3385R4] aims to support. Put differently, because there is no such thing as `^[[alignas(2)]]`, our proposed vehicle is *not* the proper way to deal with it. Hence we are not touching it.

From there the use is direct

```
struct User;
constexpr auto r = define_aggregate(^User, {
    data_member_spec(^string, {
        .name = "uuidV4",
        .attributes = { ^[[deprecated("Use UUIDV5 instead")]],
            ^[[maybe_unused]] }
    }),
    data_member_spec(^string, {
        .name = "uuidV5"
    }),
});

// Equivalent to
// struct User {
//     [[deprecated("Use UUIDV5 instead"), maybe_unused]] string uuidV4;
//     string uuidV5;
// };

```

§ 3 Proposed wording

§ 3.1 Library

- Change `data_member_options` definition to mark the current `no_unique_address` member deprecated and add the attributes data member.
- Update the *Returns:* description to accomodate for the new attributes data member.
- Update the *Effects:* to describe the new attributes data member effects.

§ 3.1.1 [meta.reflection.define.aggregate] Reflection class definition generation

```
namespace std::meta {
    struct data_member_options {
        struct name_type {
            template <typename T> requires
                constructible_from<u8string, T>
                consteval name_type(T &&);

            template <typename T> requires
                constructible_from<string, T>
                consteval name_type(T &&);
        };
    };
}
```

```
};

optional<name_type> name;
optional<int> alignment;
optional<int> bit_width;
```

```
+   vector<info> attributes;
-   bool no_unique_address = false;
+   [[deprecated]] bool no_unique_address = false;
```

```
};
}
```

...

Returns: A reflection of a data member description (T , N , A , W , NUA , AT) (11.4.1 [class.mem.general]) where

(5.1) - T is the type represented by `delias(type)`,

(5.2) - N is either the identifier encoded by `options.name` or $\perp$ if `options.name` does not contain a value,

(5.3) - A is either the alignment value held by `options.alignment` or $\perp$ if `options.alignment` does not contain a value,

(5.4) - W is either the value held by `options.bit_width` or $\perp$ if `options.bit_width` does not contain a value, and

(5.5) - NUA is the value held by `options.no_unique_address`.

+ (5.6) - AT is the value held by `options.attributes`.

...

Effects: Produces an injected declaration D ([`expr.const`]) that provides a definition for C with properties as follows:

...

If `ok.name` does not contain a value, it is an unnamed bit-field. Otherwise, it is a non-static data member with an identifier determined by the character sequence encoded by `u8identifier_of(rK)` in UTF-8.

+ Every reflection r in `ok.attributes` is appertained to D if `is_attribute(r)` is true

...

§ 4 Implementation experience

Work is underway on top of the clang p3385 ¹ branch.

§ 5 References

[P2996R10] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. 2025-02-27. Reflection for C++26.

<https://wg21.link/p2996r10>

[P3385R4] Aurelien Cassagnes. 2025-03-11. Attributes reflection.

<https://wg21.link/p3385r4>

1. <https://github.com/zebullax/clang-p2996/tree/p3385>[↩]